DATHEON / APPLICATION SECURITY

Datheon Application Security & API Pentesting Internship — Final Report

Datheon Application Security & API Pentesting Internship · 30 October–27 November 2026

Adil Firoz · Batch 01 · The Neotia University · B.Tech Computer Science and Engineering (Cybersecurity)

LOCAL SYNTHETIC-DATA TRAINING PORTFOLIO

Datheon Application Security & API Pentesting Internship — Final Report

Student: Adil Firoz · GitHub: Adilfiroz24

University: The Neotia University · Department: B.Tech Computer Science and Engineering (Cybersecurity)

Organization: Datheon · Program: Application Security & API Pentesting Internship · Batch: 01

Program dates: 30 October–27 November 2026

Executive summary

This portfolio implements four separated application-security exercises in one local TypeScript/Express training app: synthetic-data IDOR discovery, owner-authorization remediation, cookie-based session security, and a masked public profile API with security headers and CI security checks. The intentionally vulnerable behavior is isolated to an opt-in, loopback-only endpoint. No third-party system was assessed.

Objectives and scope

Demonstrate authorization testing and remediation, session design tradeoffs, data minimization and PII masking, API contract linting, automated test coverage, and baseline DAST/secret scanning. All records and account data are synthetic. Database persistence, production deployment, and remote GitHub Actions execution remain out of scope until separately configured.

Architecture and technologies

Node.js 20+, TypeScript, Express 5, Node crypto (scrypt and HMAC-SHA256), Helmet, Vitest, Supertest, Postman, cURL, Spectral, Gitleaks, OWASP ZAP, and GitHub Actions.

Assignment outcomes

1. IDOR discovery: local route discovery and controlled identifier manipulation demonstrate the risk of missing object ownership checks; secure comparison endpoint included.
2. BOLA remediation: authenticated identity and resource ownership are checked server-side; cross-user and unauthenticated regression tests included.
3. Session security: password hashing, HttpOnly cookie, short expiry, CSRF protection, throttling, rotation, and revocation are implemented. Session storage is process-local and documented as a teaching limitation.
4. API shielding: allowlisted public profile response, dynamic email/phone masking, browser security headers, OpenAPI linting, secret scanning and local-target ZAP workflow are included.

Methodology and testing

Vitest/Supertest integration tests exercise the local application with two synthetic identities. The completed local checks on 3 November 2026 included a clean npm ci, successful TypeScript build, successful Spectral lint (no error-severity findings), and npm audit with zero reported vulnerabilities. The latest automated test count is generated from the actual Vitest JSON report included in the PDF. ZAP and Gitleaks were unavailable locally and are configured in CI; their outcomes remain pending GitHub publication and workflow execution.

Evidence and risk

This is a training implementation, not a report of findings against Datheon. The illustrative CVSS vector in Task 1

describes a hypothetical exposure only. Genuine screenshots were not captured during the automated build; instructions for collecting redacted localhost evidence are provided in each task folder.

Limitations and next steps

Configure local secrets, execute and verify tests, capture genuine evidence, and run CI after repository authentication. For production-quality follow-up, replace in-memory sessions and rate limits with shared stores, use TLS and operational secret rotation, add database-backed ownership filters, and obtain an independent review.

Conclusion

The portfolio provides an auditable, reproducible localhost lab and regression tests for the four assignment themes. Claims of success, scans, or publishing must be based on the recorded local command outputs and actual GitHub workflow results.

Automated test execution

Vitest run: PASSED · 8/8 passed · 0 failed

Test case	RESULT	Duration
Task 1 and Task 2: object authorization keeps the vulnerable route disabled unless explicitly enabled	PASSED	128 ms
Task 1 and Task 2: object authorization keeps the vulnerable comparison reachable only on loopback with the explicit lab flag	PASSED	15 ms
Task 1 and Task 2: object authorization rejects unauthenticated access and prevents cross-user record reads	PASSED	76 ms
Task 3: cookie-based session security sets an HttpOnly, SameSite cookie and validates the session	PASSED	45 ms
Task 3: cookie-based session security requires CSRF validation and rotates then invalidates the prior token	PASSED	54 ms
Task 3: cookie-based session security returns a generic authentication error and applies brute-force throttling	PASSED	141 ms
Task 4: public data and response hardening masks dynamic PII and does not return the source values	PASSED	6 ms
Task 4: public data and response hardening sets baseline security headers and a restrictive CSP	PASSED	6 ms